Heapsort y la cola de prioridad

Construir un montículo en tiempo lineal, ordenar con él y atender siempre al más urgente

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Contenido

1. Repaso: el montículo de máximos
2. Construir un montículo
3. Heapsort
4. El montículo en la STL
5. El TAD cola de prioridad
6. El contenedor `priority_queue`
7. Para el juez
8. Ejercicios
9. Cierre

Contenido

1 Repaso: el montículo de máximos

2 Construir un montículo

3 Heapsort

4 El montículo en la STL

5 El TAD cola de prioridad

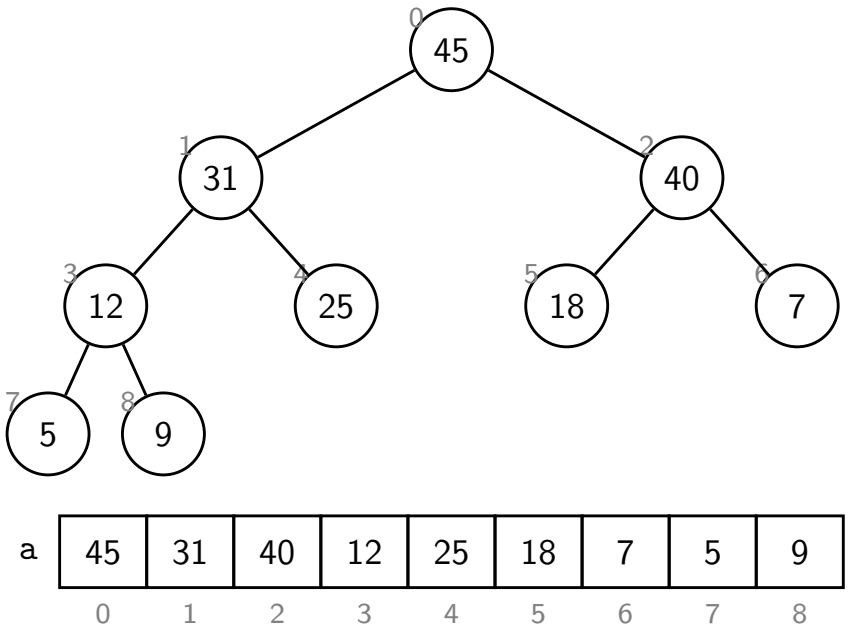
6 El contenedor `priority_queue`

7 Para el juez

8 Ejercicios

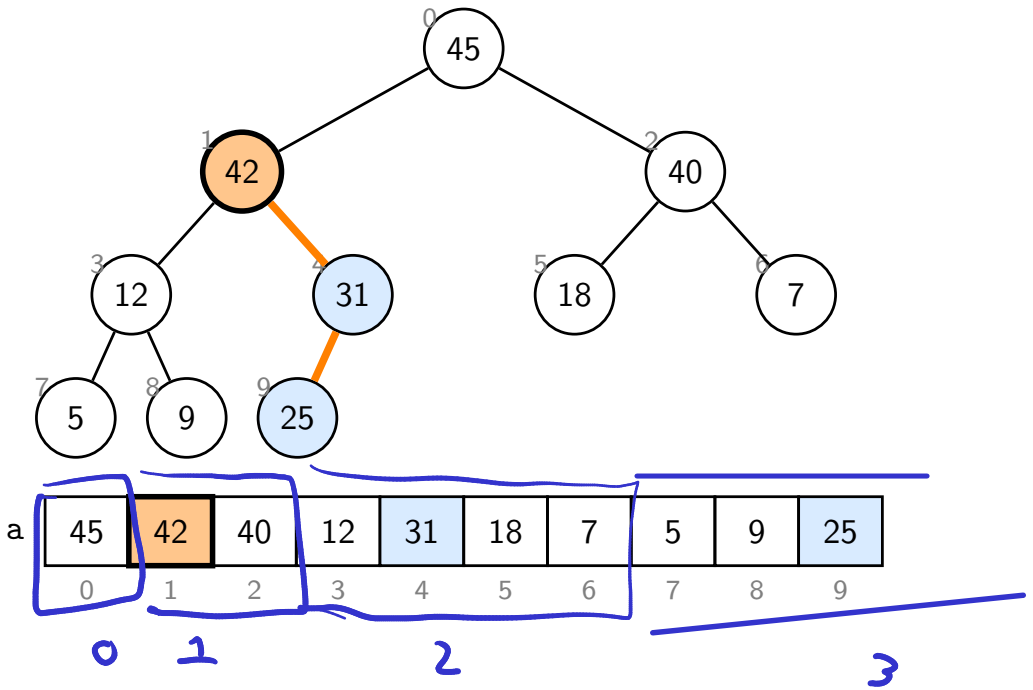
9 Cierre

El montículo de máximos



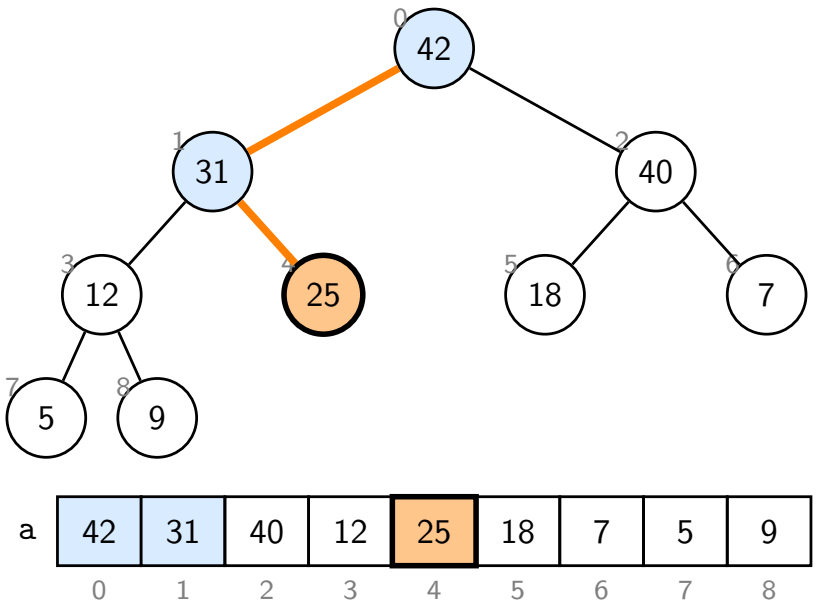
Árbol binario completo guardado en un arreglo, con cada nodo mayor o igual que sus hijos (CLRS, §6.1). Con índices desde 0, el padre de i está en $\lfloor (i-1)/2 \rfloor$ y sus hijos en $2i+1$ y $2i+2$.

Insertar: la clave sube



El 42 entra en la primera casilla libre, a[9], y sube mientras su padre sea menor: pasa por encima del 25 y del 31. A lo sumo $\lfloor \log_2 n \rfloor$ intercambios: $O(\log n)$.

Extraer el máximo: la clave baja



Sale el 45, la última clave (el 25) pasa a la raíz y baja hacia el mayor de sus hijos: primero el 42, después el 31. También $O(\log n)$, y consultar el máximo es leer `a[0]`, $\Theta(1)$.

Contenido

1 Repaso: el montículo de máximos

2 Construir un montículo

3 Heapsort

4 El montículo en la STL

5 El TAD cola de prioridad

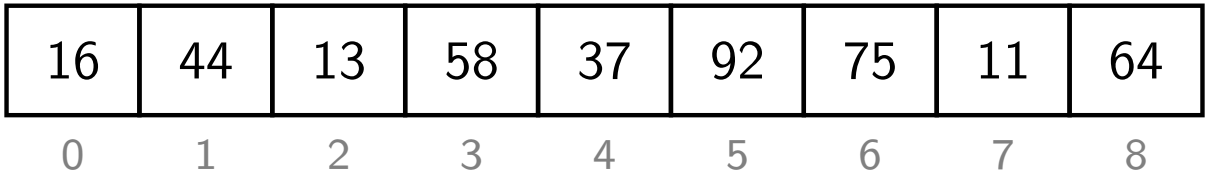
6 El contenedor `priority_queue`

7 Para el juez

8 Ejercicios

9 Cierre

Un arreglo cualquiera



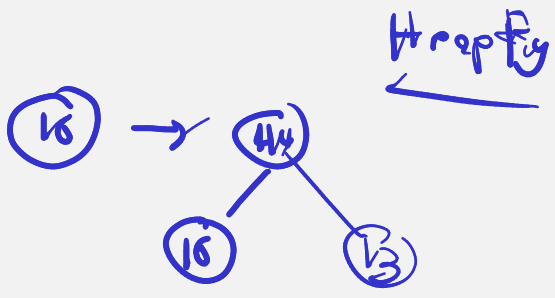
Pregunta

Llegan estas nueve claves, sin ningún orden. ¿Cómo se convierten en un montículo de máximos, dentro del mismo arreglo?

Primera idea: insertar una por una

Cada casilla, de la 1 en adelante, se sube como si acabara de llegar.
El prefijo `a[0..i-1]` ya es un montículo cuando le toca a `a[i]`.

```
// Versión ingenua: inserta las claves una por una.
void construirSubiendo(vector<int> &a) {
  int i = 1;
  while (i < (int) a.size()) {
    subir(a, i);
    i = i + 1;
  }
}
```



Son $n - 1$ llamados a `subir`, cada uno de $O(\log n)$: $O(n \log n)$.



Primera idea: insertar una por una

El peor caso

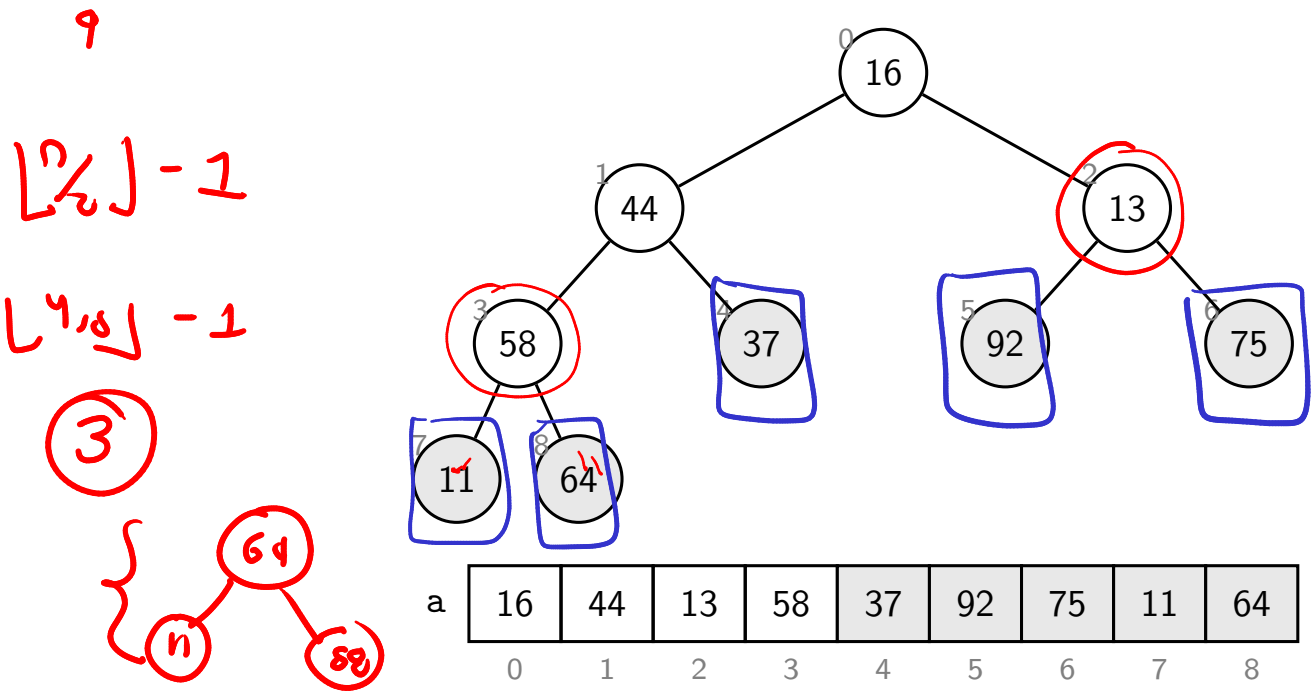
Si las claves llegan en orden creciente, cada una es la mayor hasta ese momento y sube hasta la raíz. En un árbol de 15 claves, las 8 del último nivel suben 3 niveles cada una: casi la mitad de las claves hace el recorrido más largo.

Primera idea: insertar una por una

Pregunta

¿Se puede trabajar al revés, de modo que las claves que hacen el recorrido largo sean pocas?

Las hojas ya son montículos



Las casillas grises no tienen hijos: cada una es, sola, un montículo de una clave. Falta arreglar las de arriba, y para eso está `bajar`.

¿Desde qué casilla se empieza?

Pregunta

Con $n = 9$, ¿cuál es la última casilla que tiene al menos un hijo? ¿Y con n claves?

¿Desde qué casilla se empieza?

Respuesta

La casilla 3: su hijo izquierdo es $2 \cdot 3 + 1 = 7$, que existe. El de la casilla 4 sería la 9, fuera del arreglo. Con n claves, la última casilla con hijos es $\lfloor n/2 \rfloor - 1$, y de $\lfloor n/2 \rfloor$ en adelante todas son hojas (CLRS, §6.3).

bajar con el tamaño aparte

```
// Baja la clave de i dentro de a[0..n-1].
void bajar(vector<int> &a, int n, int i) {
  bool sigue = true;
  while (sigue) {
    int izq = 2 * i + 1;
    int der = 2 * i + 2;
    int mayor = i;
    if (izq < n && a[izq] > a[mayor]) {
      mayor = izq;
    }
    if (der < n && a[der] > a[mayor]) {
      mayor = der;
    }
    if (mayor != i) {
      intercambiar(a, i, mayor);
      i = mayor;
    } else {
      sigue = false;
    }
  }
```



`bajar` con el tamaño aparte

Es el `bajar` de `monticulo.h`, ahora como función sobre cualquier `vector`. El parámetro `n` dice cuántas casillas del principio forman el montículo: las de `n` en adelante no se miran. `construir` lo llama con el tamaño completo; heapsort, con uno que se acorta en cada ronda.

construir: bajar de abajo arriba

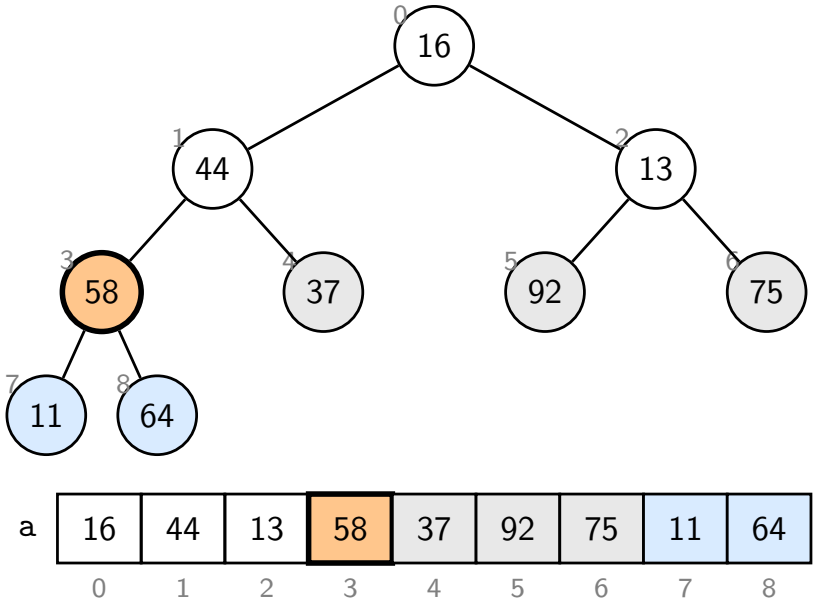
```
// Convierte un arreglo cualquiera en montículo de máximos.
void construir(vector<int> &a) {
  int n = (int) a.size();
  int i = n / 2 - 1;
  while (i >= 0) {
    bajar(a, n, i);
    i = i - 1;
  }
}
```



Lo que se cumple en cada vuelta

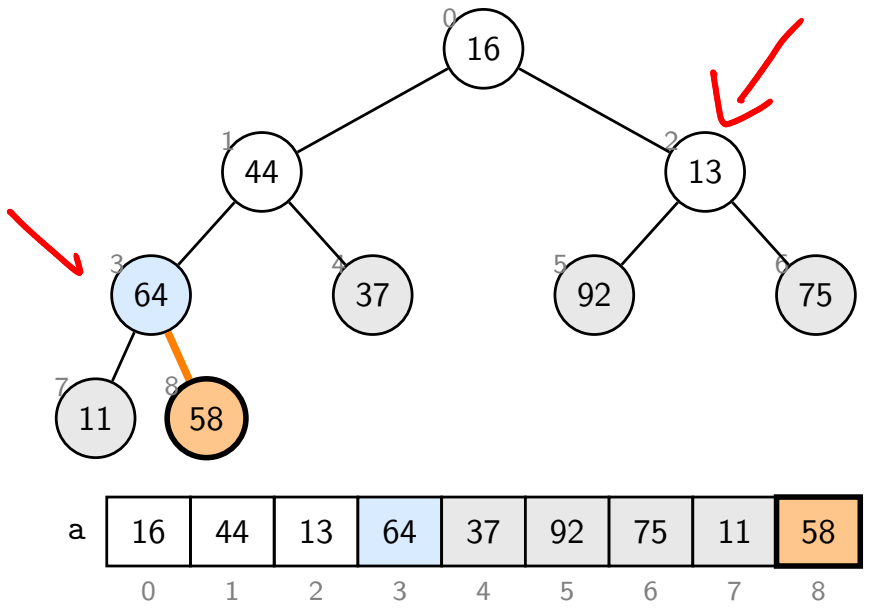
Cuando le toca a la casilla i, toda casilla mayor que i ya es la raíz de un montículo. Los dos hijos de i lo son, y eso es lo que `bajar` necesita para dejar un montículo con raíz en i.

construir: traza



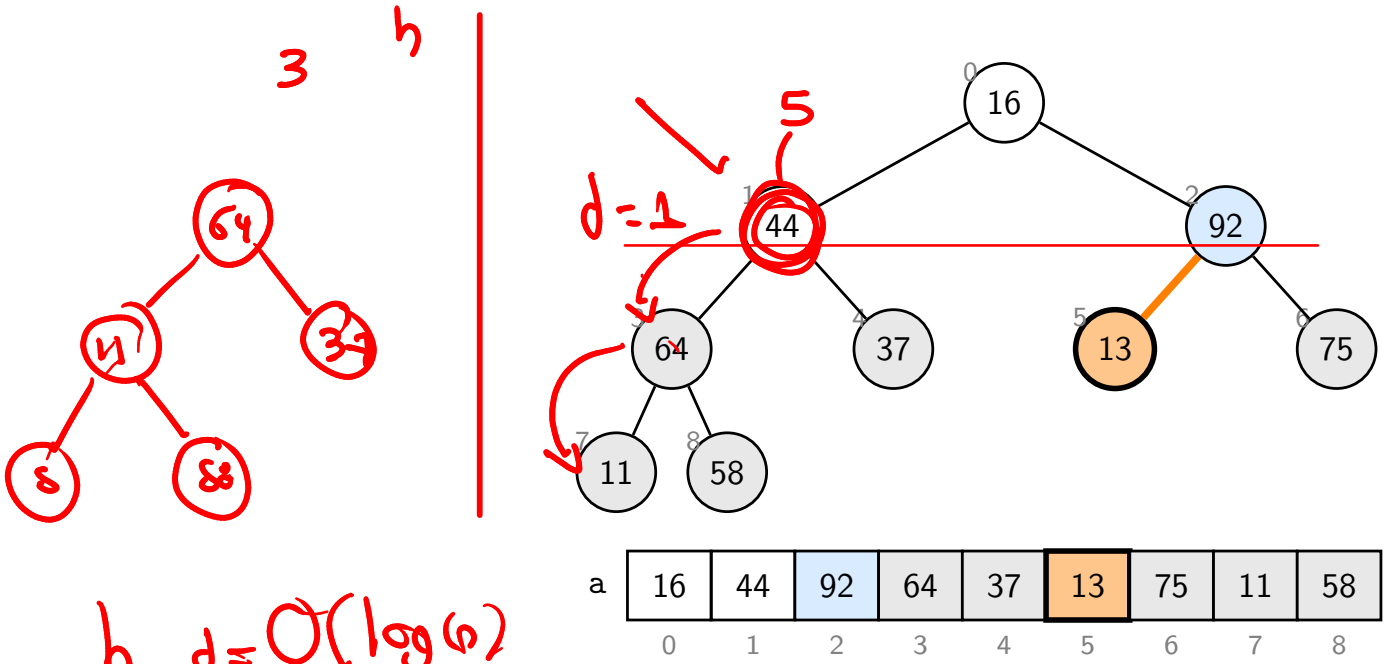
Empieza en $i = 3$: el 58 se compara con sus hijos, 11 y 64.

construir: traza



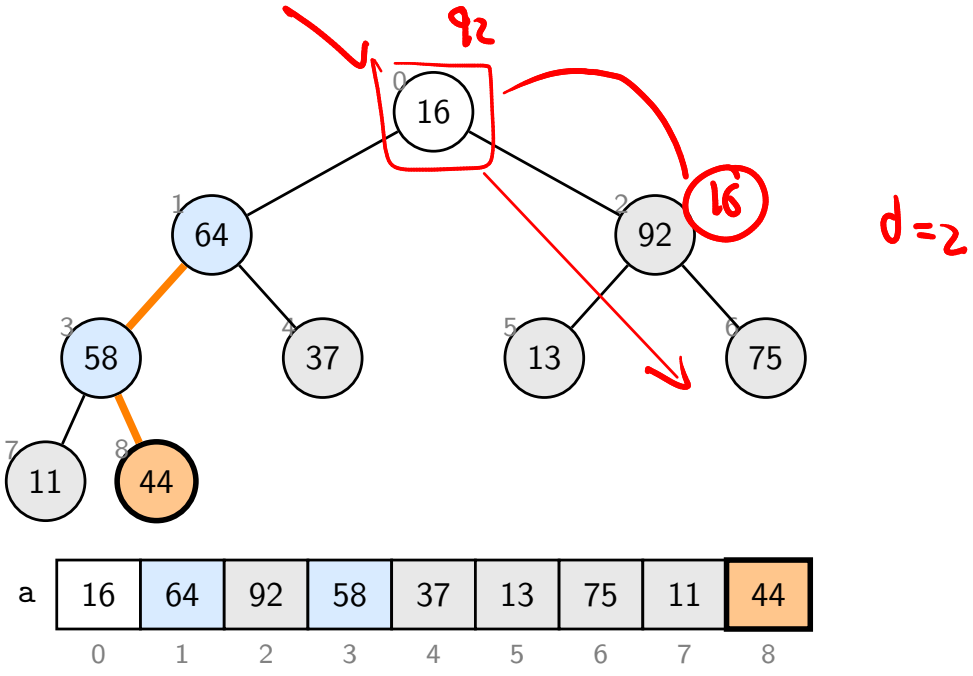
llamado	clave	cambia con	intercambios
`bajar(3)`	58	64	1

construir: traza



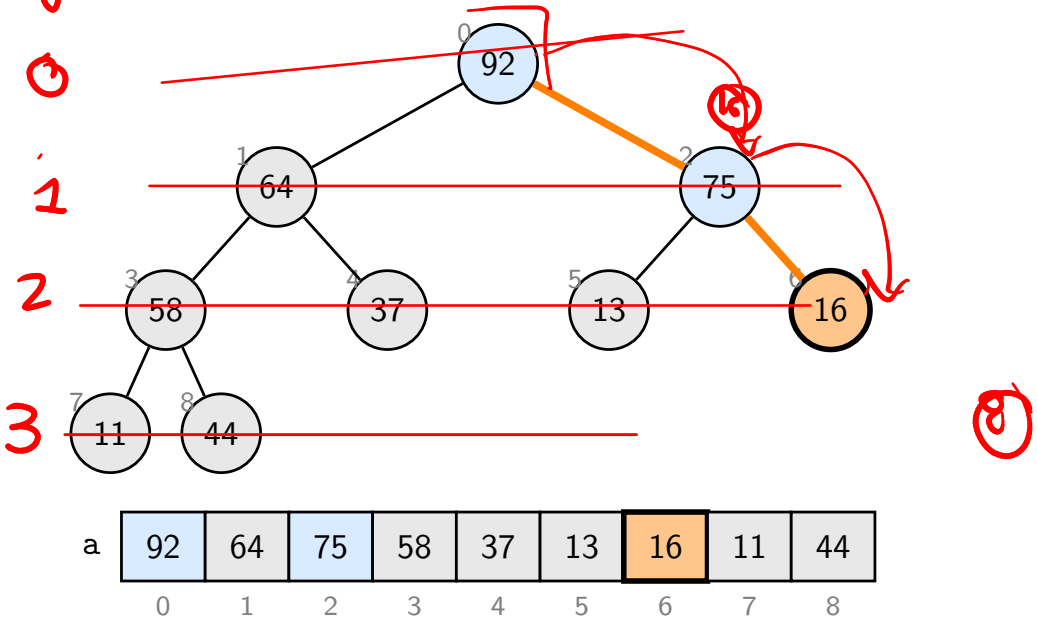
llamado	clave	cambia con	intercambios
`bajar(3)`	58	64	1
`bajar(2)`	13	92	1

construir: traza



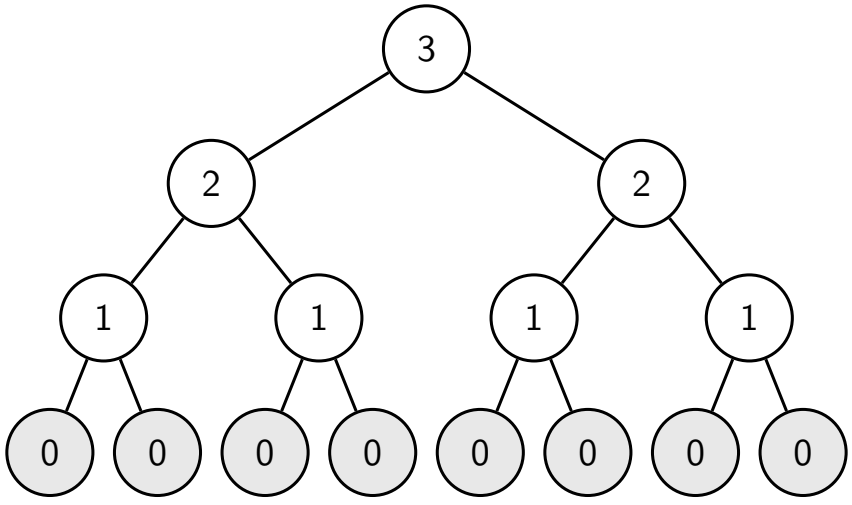
llamado	clave	cambia con	intercambios
`bajar(3)`	58	64	1
`bajar(2)`	13	92	1
`bajar(1)`	44	64, luego 58	2

construir: traza



llamado	clave	cambia con	intercambios
`bajar(3)`	58	64	1
`bajar(2)`	13	92	1
`bajar(1)`	44	64, luego 58	2
`bajar(0)`	16	92, luego 75	2

Cuánto puede bajar cada casilla



Con 15 claves, cada nodo muestra cuántos niveles puede bajar. Bajando: $4 \cdot 1 + 2 \cdot 2 + 1 \cdot 3 = 11$ intercambios a lo sumo. Subiendo, una clave del nivel d sube d: $2 \cdot 1 + 4 \cdot 2 + 8 \cdot 3 = 34$.

`construir` cuesta $\Theta(n)$

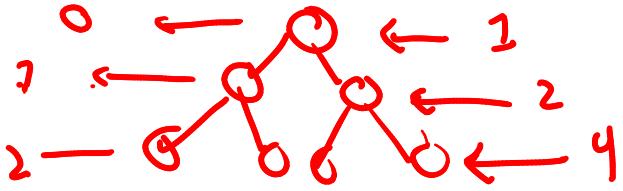
Teorema (CLRS, §6.3)

Sobre un arreglo de $n \geq 1$ claves, `construir` hace a lo sumo $2n$ intercambios.

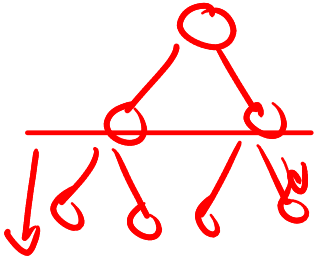
Demostración

Se procede de forma directa, contando los intercambios por niveles del árbol.

construir cuesta $\Theta(n)$: desarrollo



Sea $h = \lfloor \log_2 n \rfloor$ la altura. El nivel d tiene a lo sumo 2^d casillas, y una clave que empieza en el nivel d baja a lo sumo $h - d$ niveles, con un intercambio por nivel:



$$I(n) \leq \sum_{d=0}^{h} 2^d (h-d) = \sum_{j=0}^{h} 2^{h-j} j = 2^h \sum_{j=0}^{h} \frac{j}{2^j}$$

La igualdad del medio cambia d por $j = h - d$, que recorre los mismos valores de 0 a h.

$$\sum_{d=0}^{h} 2^d (h-d) = \sum_{j=0}^{h} 2^{h-j} \times j = \sum_{j=0}^{h} 2^h \times \frac{j}{2^j}$$

$j = (h-d)$

$d = h-j$

$$2^h \sum_{j=0}^{h} \frac{j}{2^j}$$

`construir` cuesta $\Theta(n)$: desarrollo

Sea $S = \sum_{j \geq 1} j/2^j$. Restando la mitad de S término a término:

$$S - \frac{S}{2} = \sum_{j \geq 1} \frac{j}{2^j} - \sum_{j \geq 2} \frac{j-1}{2^j} = \frac{1}{2} + \sum_{j \geq 2} \frac{1}{2^j} = \sum_{j \geq 1} \frac{1}{2^j} = 1$$

Entonces $S = 2$. La suma hasta h tiene solo términos positivos, así que no pasa de 2, y como $2^h \leq n$:

$$I(n) \;\leq\; 2^h \cdot 2 \;\leq\; 2n.$$

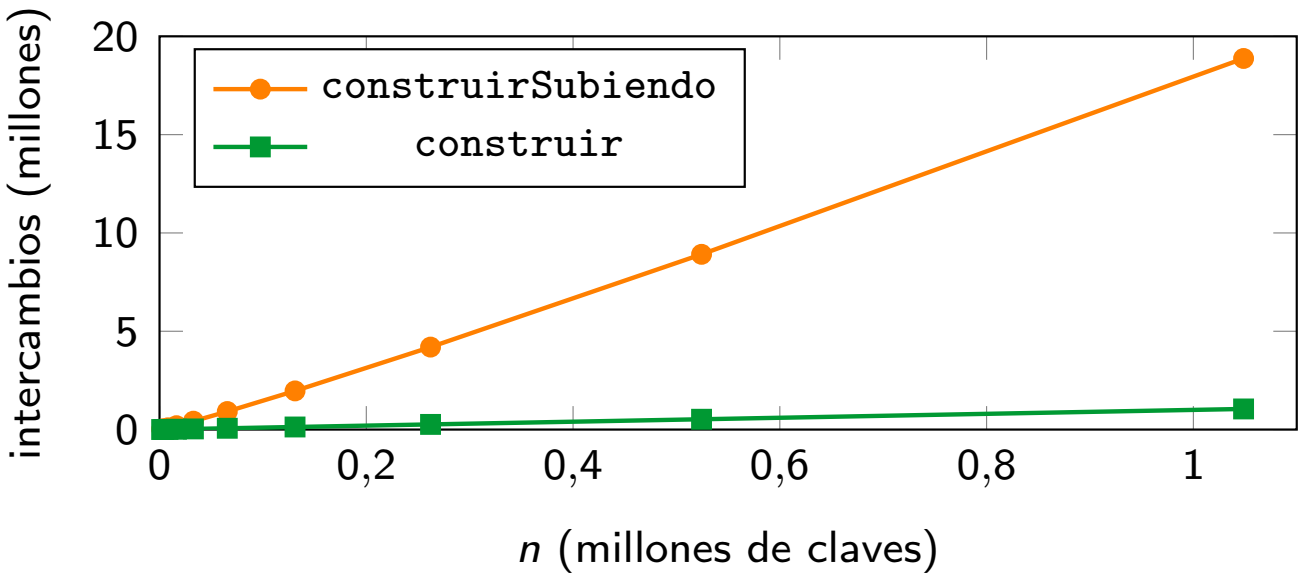
`construir` cuesta $\Theta(n)$: conclusión

Conclusión

Por lo tanto, se puede concluir que $I(n) \leq 2n$ para todo $n \geq 1$, es decir, $I(n) \in O(n)$ con testigos $c = 2$ y $k = 1$.

Cada vuelta de `bajar` hace dos comparaciones, y hay una vuelta por intercambio más la última de cada llamado. Las comparaciones son a lo sumo $2(2n + n/2) = 5n$: tiempo $O(n)$ con $c = 5$ y $k = 1$. El ciclo visita $\lfloor n/2 \rfloor$ casillas, así que tampoco baja de lineal: $\Theta(n)$.

Medido: el peor caso de los dos caminos



Claves $1, 2, \ldots, n$ en orden creciente. Con $n = 1\,048\,575$: $1\,048\,555$ intercambios bajando y $18\,874\,370$ subiendo, 18 veces más.

Lo que cuesta construir

Operación	Costo	Por qué
`construirSubiendo`	$O(n \log n)$	cada clave sube hasta $\lfloor \log_2 n \rfloor$ niveles, y la mitad está en el último
`construir`	$\Theta(n)$	la mitad son hojas y no bajan; las que bajan mucho son pocas (teorema)
espacio aparte, los dos	$\Theta(1)$	trabajan dentro del mismo arreglo, con unas pocas variables

Contenido

1 Repaso: el montículo de máximos

2 Construir un montículo

3 Heapsort

4 El montículo en la STL

5 El TAD cola de prioridad

6 El contenedor `priority_queue`

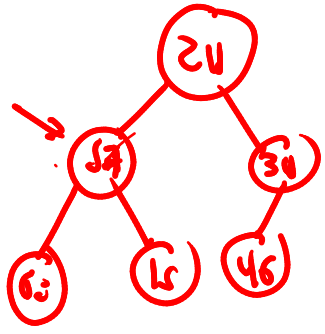
7 Para el juez

8 Ejercicios

9 Cierre

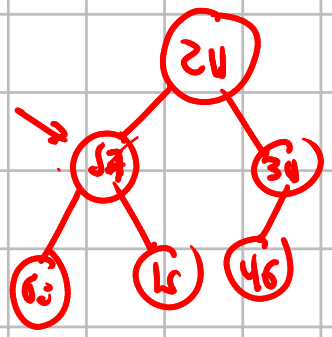
Ordenar con el montículo

24	57	34	68	15	46
0	1	2	3	4	5

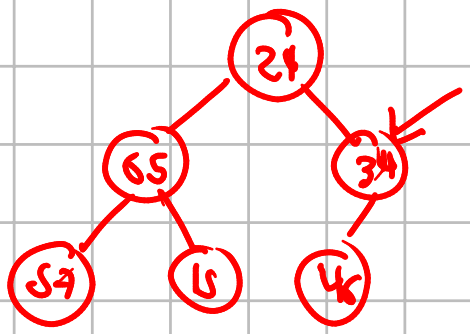


Primera idea

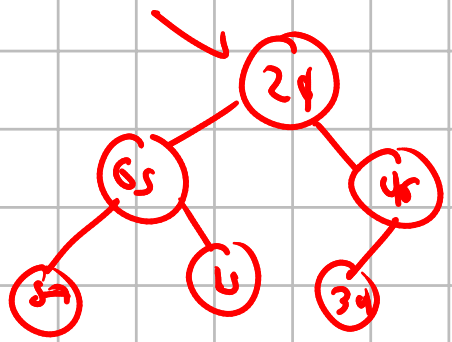
`construir`, y después extraer el máximo seis veces, poniendo cada uno en otro arreglo desde el final. Cuesta $O(n \log n)$, pero pide un segundo arreglo de n casillas: $\Theta(n)$ de espacio aparte.



24 57 34 65 15 46



24 65 34 57 15 46



24 65 46 57 15 34

65 57 46 24 11 34

34	57	46	24	11	X 65
0	1	2	3	4	5

57 34 46 24 11 | 65

15 34 46 25 | 57 65

46 34 15 25 | 57 65

25 34 15 | 46 57 65

34 25 18 | 48 57 65

15 25 | 34 48 57 65

25 15 | 34 48 57 65

(15) 15 | 25 34 46 57 65

La casilla que se libera

Pregunta

Al extraer el máximo, el montículo pierde su última casilla, `a[fin]`. En el arreglo ordenado de menor a mayor, ¿qué clave le corresponde a esa casilla?

La casilla que se libera

Respuesta

El máximo de lo que queda en el montículo: es el mayor de las $\mathit{fin} + 1$ claves que faltan por ordenar. Entonces no hace falta otro arreglo. Se cambia `a[0]` con `a[fin]`, el montículo se acorta en una casilla y la clave que llegó a la raíz baja (CLRS, §6.4).

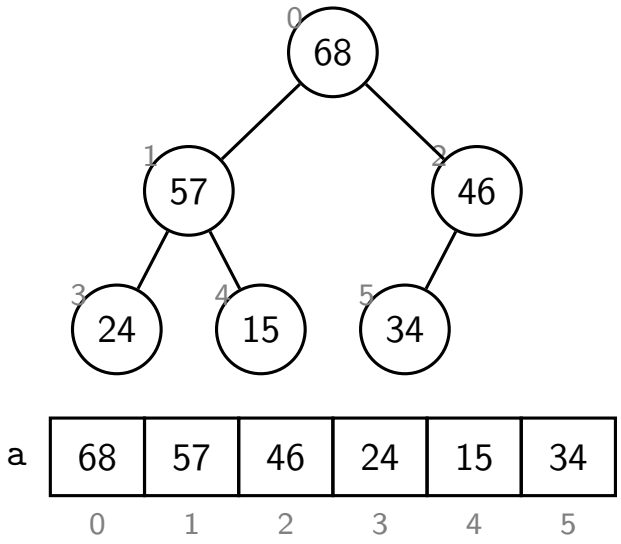
heapsort

```
// Ordena de menor a mayor sin arreglo auxiliar.
void heapsort(vector<int> &a) {
  construir(a);
  int fin = (int) a.size() - 1;
  while (fin > 0) {
    intercambiar(a, 0, fin);
    bajar(a, fin, 0);
    fin = fin - 1;
  }
}
```

Lo que se cumple en cada vuelta

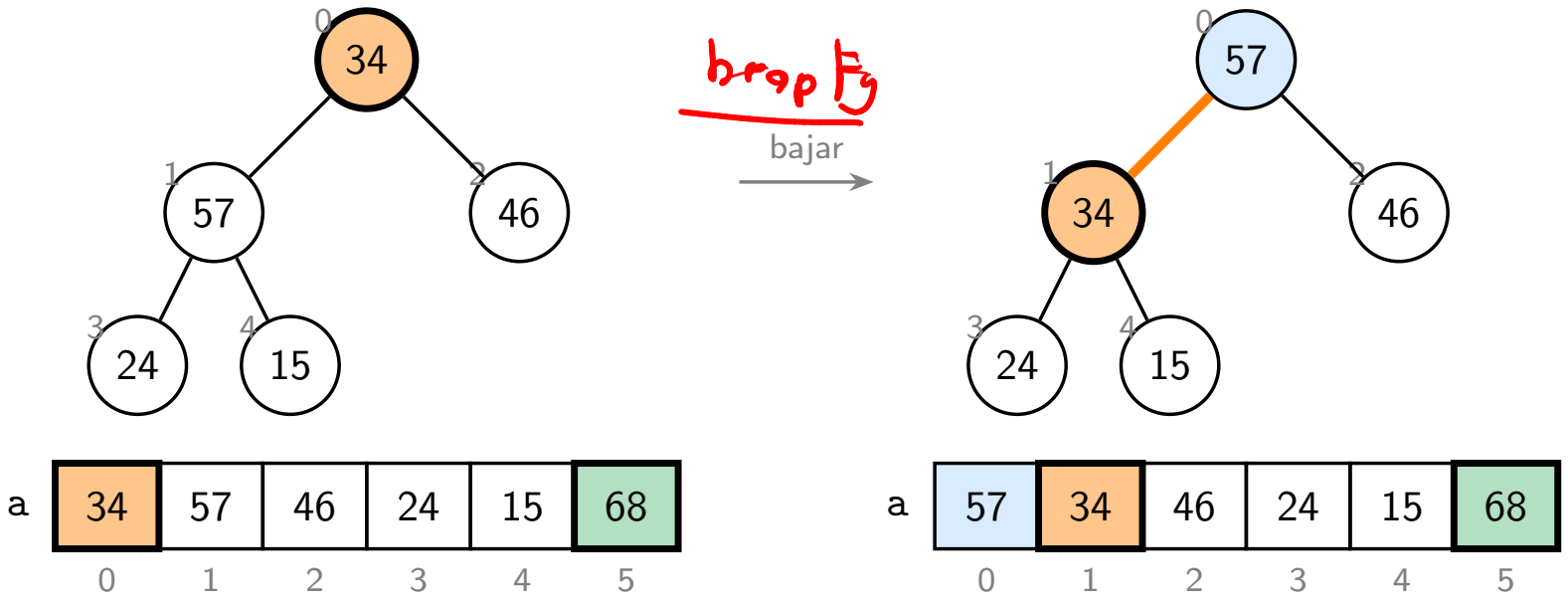
`a[0..fin]` es un montículo con las claves menores, y `a[fin+1..n-1]` tiene las mayores, ya en orden. `bajar` recibe `fin` como tamaño y no toca la parte ordenada.

heapsort: traza



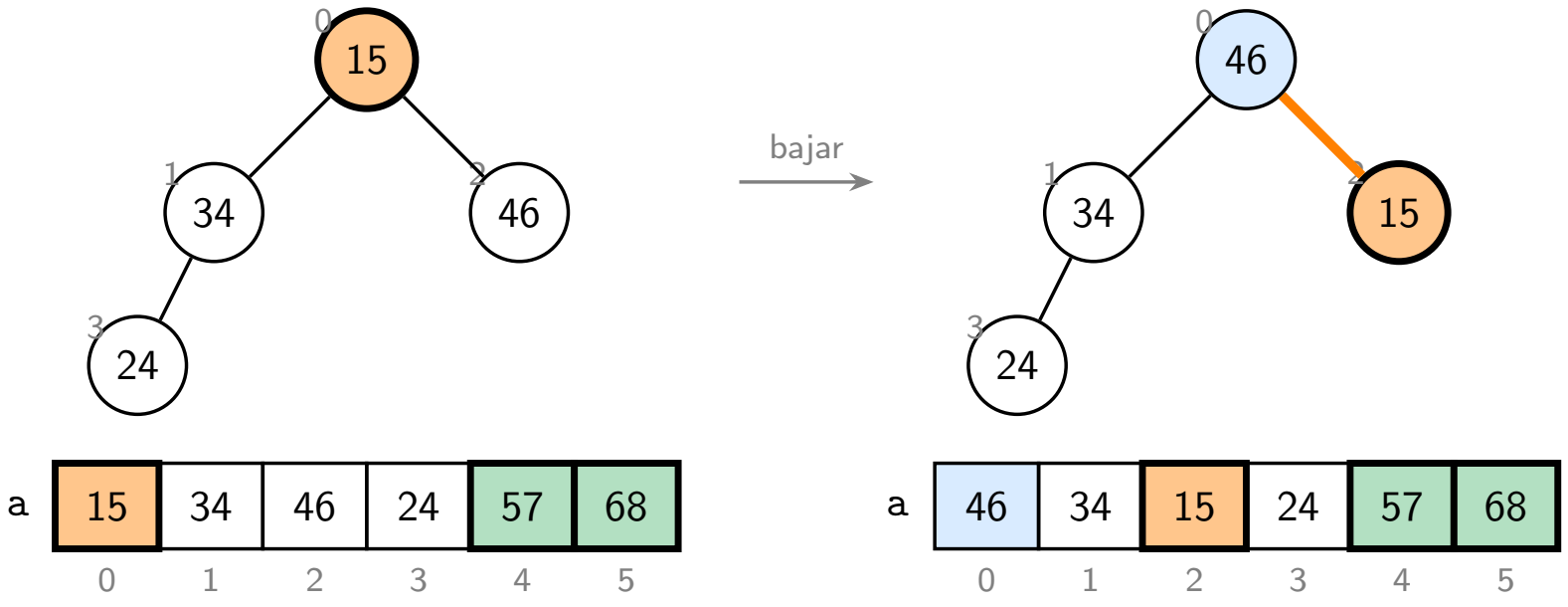
`construir` deja el montículo 68 57 46 24 15 34. Empieza la ronda con $fin = 5$.

heapsort: traza



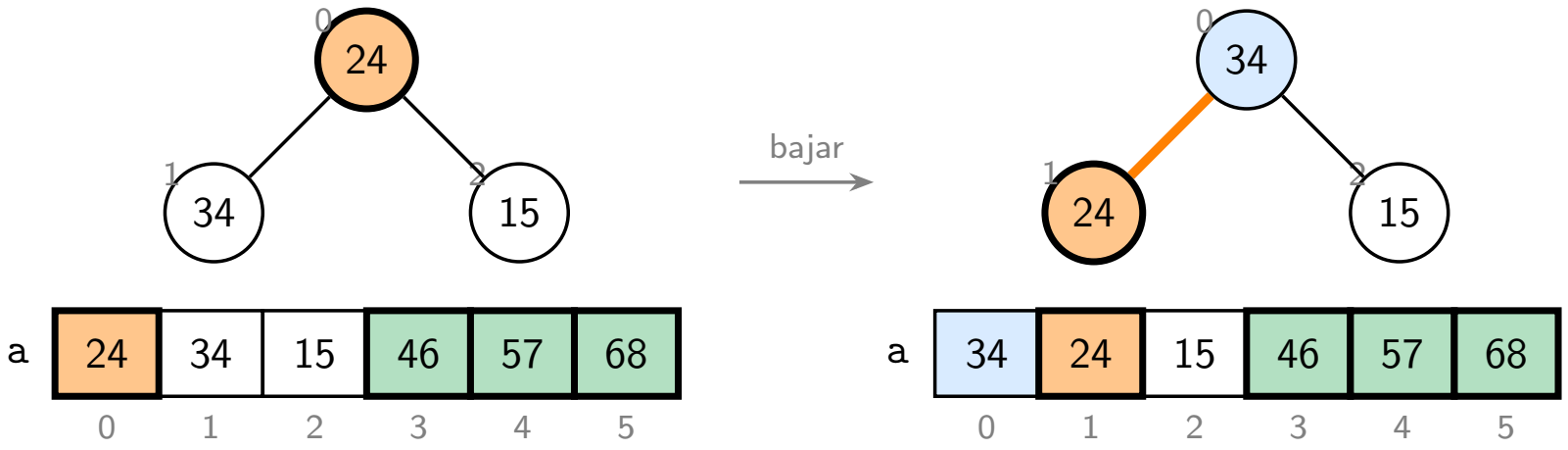
fin = 5: el 68 pasa a a[5] y el 34 llega a la raíz. Baja por el 57, el mayor de sus hijos, y se detiene sobre el 24 y el 15.

heapsort: traza



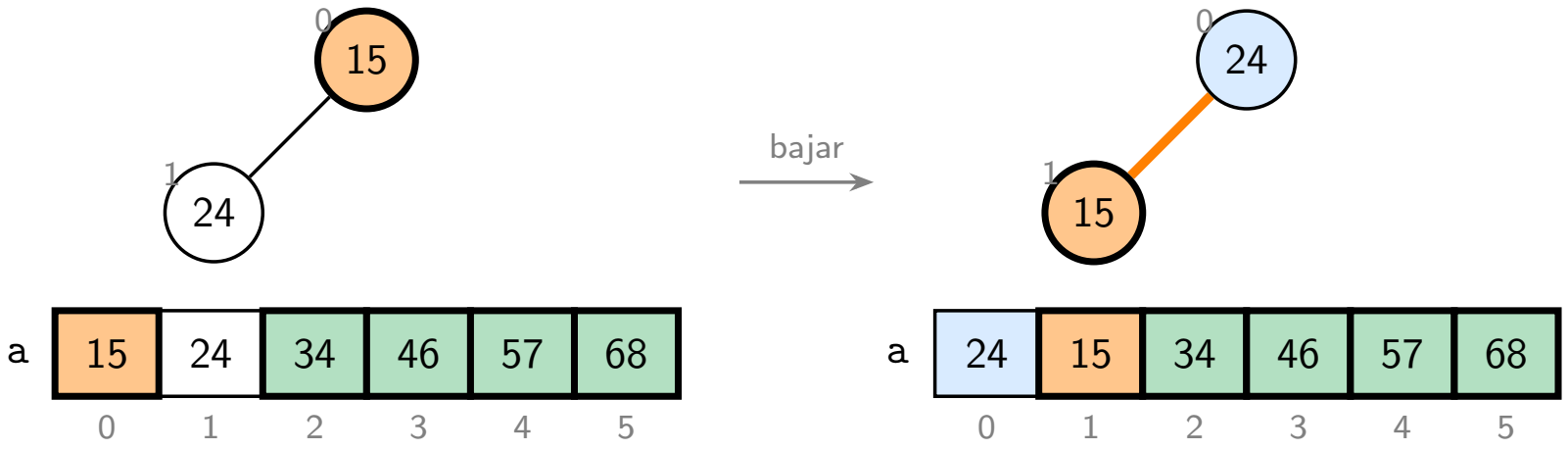
fin $= 4$: el 57 pasa a a[4]. El 15 baja por el 46, que es mayor que el 34, y queda en una hoja.

heapsort: traza



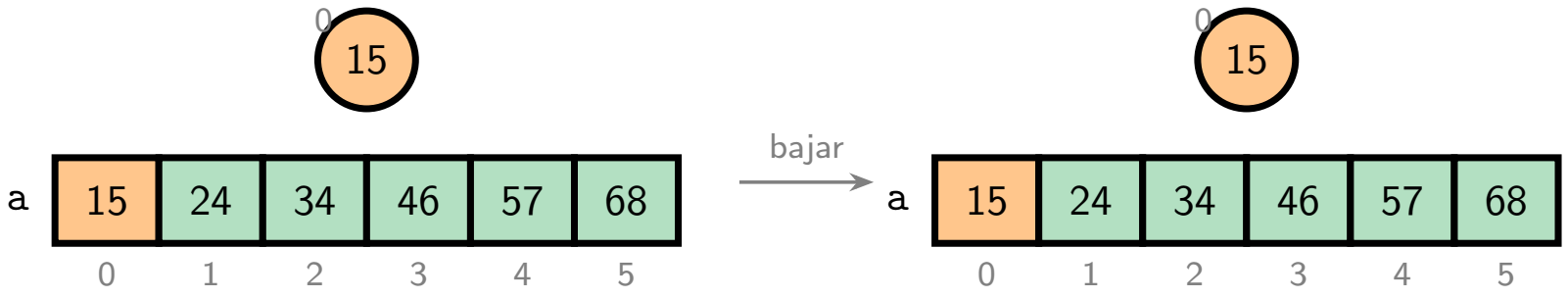
fin = 3: el 46 pasa a a[3]. El 24 baja por el 34.

heapsort: traza



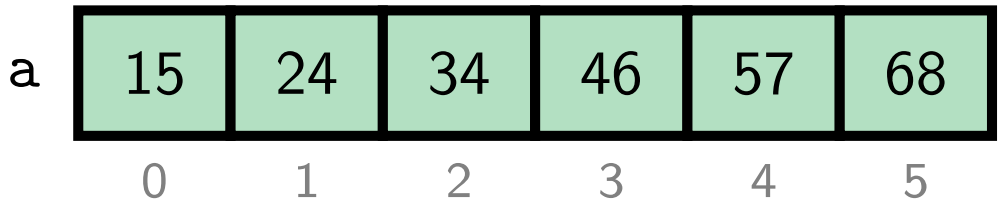
fin $= 2$: el 34 pasa a a[2]. El 15 baja por el 24, su único hijo dentro del montículo.

heapsort: traza



fin $= 1$: el 24 pasa a `a[1]`. Queda el 15 solo, sin nada que bajar.

heapsort: traza



Con $fin = 0$ el ciclo termina: `15 24 34 46 57 68`. Fueron cinco rondas para seis claves, y la menor quedó en `a[0]` sin moverla.

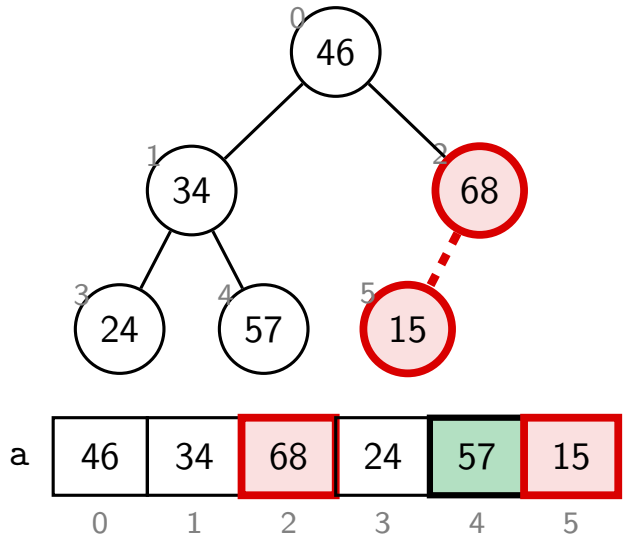
Bajar sin acortar el montículo

Pregunta

Esta versión llama a `bajar` con el tamaño completo. Sobre `24 57 34 68 15 46`, ¿qué pasa en la segunda ronda?

```
void heapsortSinAcortar(vector<int> &a) {
  construir(a);
  int n = (int) a.size();
  int fin = n - 1;
  while (fin > 0) {
    intercambiar(a, 0, fin);
    bajar(a, n, 0);
    fin = fin - 1;
  }
}
```

Bajar sin acortar el montículo



En la segunda ronda el 15 llega a la raíz y baja por el 46 hasta `a[2]`. Ahí `bajar` todavía ve a `a[5]`, el 68 que ya estaba en su lugar, y lo sube. El arreglo termina en 68 57 24 46 34 15, sin ordenar.

Lo que cuesta heapsort

Parte	Costo	Por qué
`construir`	$\Theta(n)$	el teorema de la sección anterior
una ronda	$O(\log n)$	un intercambio y una bajada de a lo sumo $\lfloor \log_2 n \rfloor$ niveles
las $n-1$ rondas	$O(n \log n)$	también en el peor caso: la cota de cada ronda vale para cualquier orden de llegada
espacio aparte	$\Theta(1)$	ordena dentro del mismo arreglo, y `bajar` es un ciclo, sin recursión
estable	no	el cambio de `a[0]` con `a[fin]` salta por encima de lo que hay en medio, y dos claves iguales pueden quedar invertidas

Contenido

1 Repaso: el montículo de máximos

2 Construir un montículo

3 Heapsort

4 El montículo en la STL

5 El TAD cola de prioridad

6 El contenedor `priority_queue`

7 Para el juez

8 Ejercicios

9 Cierre

Cinco funciones de <algorithm>

Trabajan sobre un rango de un `vector`, dado por dos iteradores, y lo mantienen en orden de montículo de máximos.

Función	Lo que hace, con lo que se vio en clase
`make_heap(b, e)`	`construir`: baja cada casilla con hijos, desde $n/2 - 1$ hasta la raíz
`push_heap(b, e)`	`subir` la última casilla del rango
`pop_heap(b, e)`	una ronda de heapsort: cambia la raíz con la última casilla y baja la nueva raíz en un rango una casilla más corto
`sort_heap(b, e)`	el ciclo de heapsort, sobre un rango que ya es montículo
`is_heap(b, e)`	revisa cada casilla contra su padre

Cinco funciones de <algorithm>

No agregan ni quitan casillas

El tamaño del `vector` lo cambian `push_back` y `pop_back`, y el orden de las llamadas importa:

- para agregar, primero `push_back` y después `push_heap`, que sube la clave que quedó al final;
- para sacar, primero `pop_heap`, que deja el máximo al final, y después `pop_back`, que lo quita.

Las funciones en uso

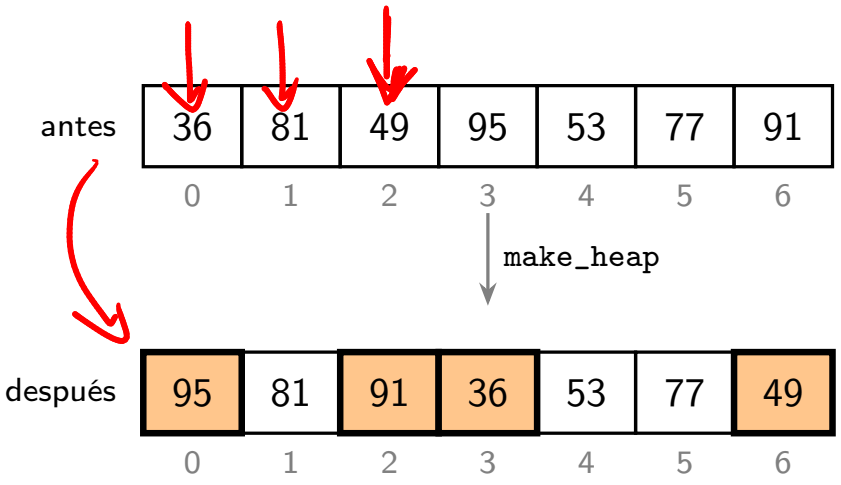
```
// Arma el montículo y agrega una clave.
void armarYAgregar(vector<int> &v) {
  imprimir("arreglo:", v);
  printf("is_heap: %d\n", (int) is_heap(v.begin(), v.end()));
  make_heap(v.begin(), v.end());
  imprimir("make_heap:", v);
  printf("is_heap: %d\n", (int) is_heap(v.begin(), v.end()));
  v.push_back(88);
  imprimir("push_back(88):", v);
  push_heap(v.begin(), v.end());
  imprimir("push_heap:", v);
}
```

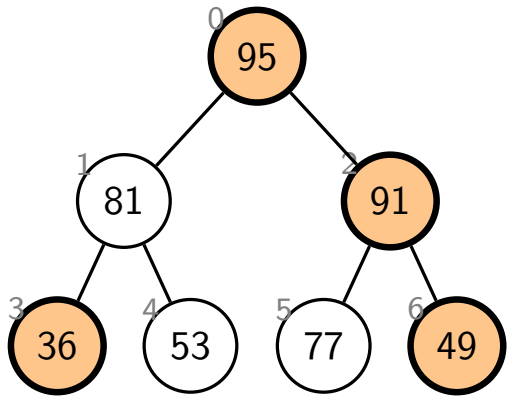
Las funciones en uso

```
// Saca el máximo y ordena lo que queda.
void sacarYOrdenar(vector<int> &v) {
  pop_heap(v.begin(), v.end());
  imprimir("pop_heap:", v);
  printf("back() = %d\n", v.back());
  v.pop_back();
  imprimir("pop_back:", v);
  sort_heap(v.begin(), v.end());
  imprimir("sort_heap:", v);
  printf("is_heap: %d\n", (int) is_heap(v.begin(), v.end()));
}
```

Las dos funciones reciben el mismo `vector` por referencia, con las claves 36 81 49 95 53 77 91.

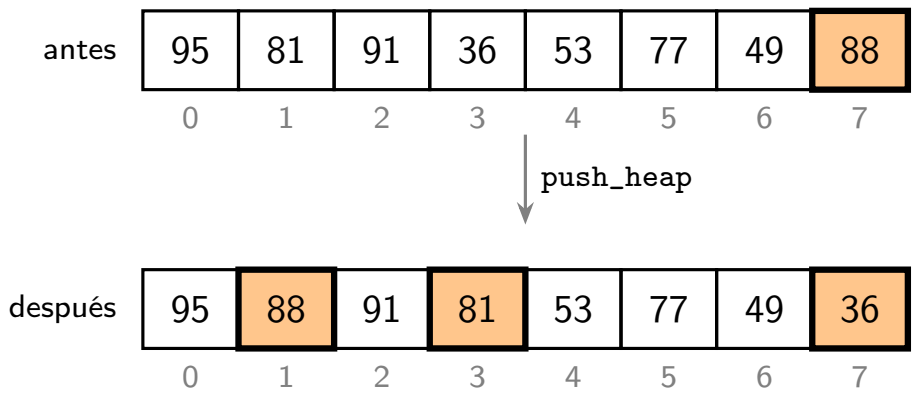
El vector, llamada por llamada

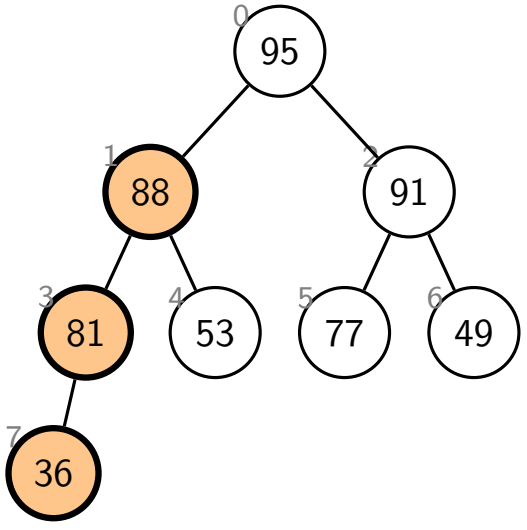




`make_heap` deja 95 81 91 36 53 77 49, el mismo arreglo que da `construir`. `is_heap` pasa de 0 a 1.

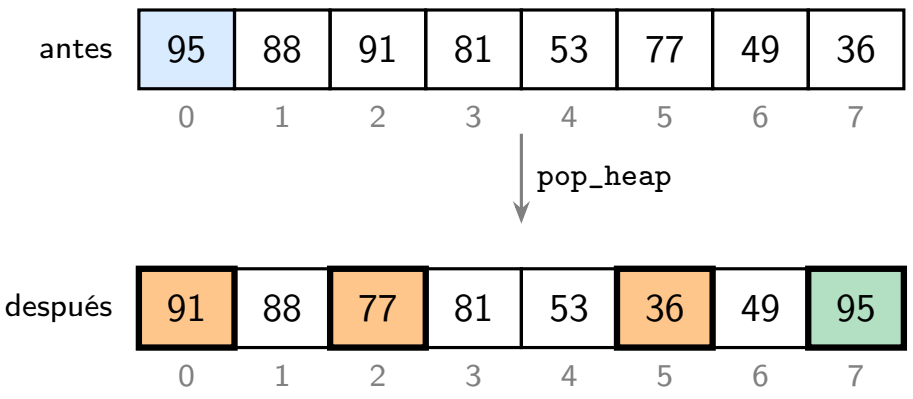
El vector, llamada por llamada

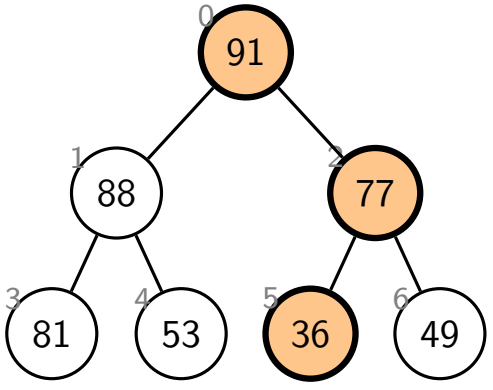




`push_back(88)` pone el 88 en `v[7]`, y `push_heap` lo sube por encima del 36 y del 81, como `subir`.

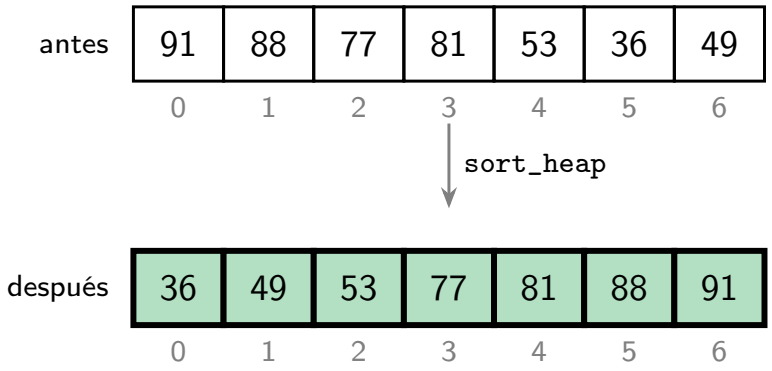
El vector, llamada por llamada





`pop_heap` cambia el 95 con el 36 y baja el 36 en las siete primeras casillas. El 95 queda en `v[7]`, fuera del montículo, y `back()` lo devuelve antes del `pop_back`.

El vector, llamada por llamada



Tras el `pop_back` quedan siete claves, y `sort_heap` las ordena de menor a mayor. El resultado ya no es un montículo de máximos: `is_heap` da 0.

El orden de las llamadas

Pregunta

Con el vector en `95 88 91 81 53 77 49 36`, alguien llama `pop_back` antes de `pop_heap`. ¿Qué clave sale?

El orden de las llamadas

Respuesta

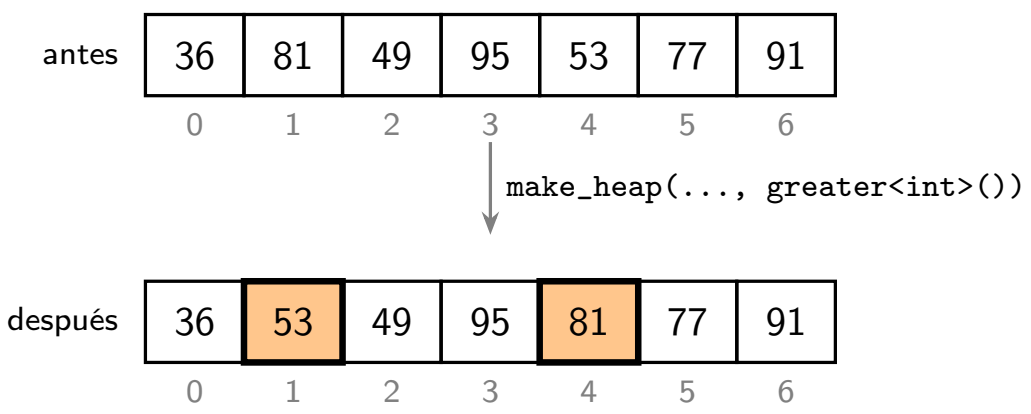
Sale el 36, que estaba en la última casilla, y el 95 se queda. Quitar una hoja no rompe el montículo, así que nada lo delata: el programa sigue con la clave equivocada. Y si se llama `push_heap` antes del `push_back`, la clave nueva no está en el rango y no sube.

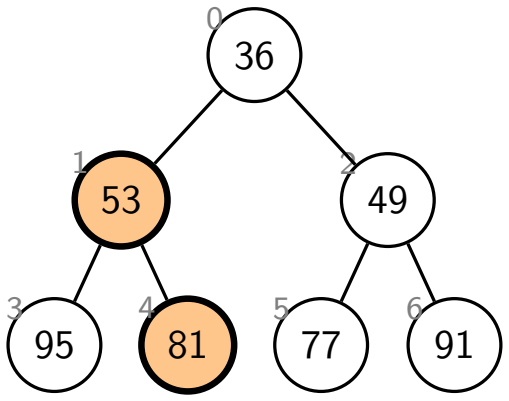
El de mínimos con greater<int>()

```cpp
// Con greater<int>() el menor queda en w[0].
void minimos() {
  int claves[7] = {36, 81, 49, 95, 53, 77, 91};
  vector<int> w(claves, claves + 7);
  make_heap(w.begin(), w.end(), greater<int>());
  imprimir("min make_heap:", w);
  pop_heap(w.begin(), w.end(), greater<int>());
  printf("sale el menor: %d\n", w.back());
  w.pop_back();
  imprimir("min pop_back:", w);
}
```

`greater<int>()`, de `<functional>`, compara con `a > b`, como `MenorArriba`. Cada llamada sobre ese rango lleva el mismo comparador.

El de mínimos con `greater<int>()`





Queda `36 53 49 95 81 77 91`, con el 36 en `w[0]`. `pop_heap` con el mismo comparador lo lleva al final, y quedan `49 53 77 95 81 91`.

Lo que cuesta el montículo de la STL

Función	Costo	Por qué
`make_heap`	$\Theta(n)$	a lo sumo $3n$ comparaciones, por el mismo argumento de `construir`
`push_heap`	$O(\log n)$	la última clave sube por un camino de largo $\lfloor \log_2 n \rfloor$
`pop_heap`	$O(\log n)$	un intercambio y una bajada
`sort_heap`	$O(n \log n)$	$n - 1$ veces lo de `pop_heap`
`is_heap`	$O(n)$	cada casilla contra su padre, hasta la primera que falle
espacio aparte	$\Theta(1)$	todo pasa dentro del mismo `vector`

Contenido

1 Repaso: el montículo de máximos

2 Construir un montículo

3 Heapsort

4 El montículo en la STL

5 El TAD cola de prioridad

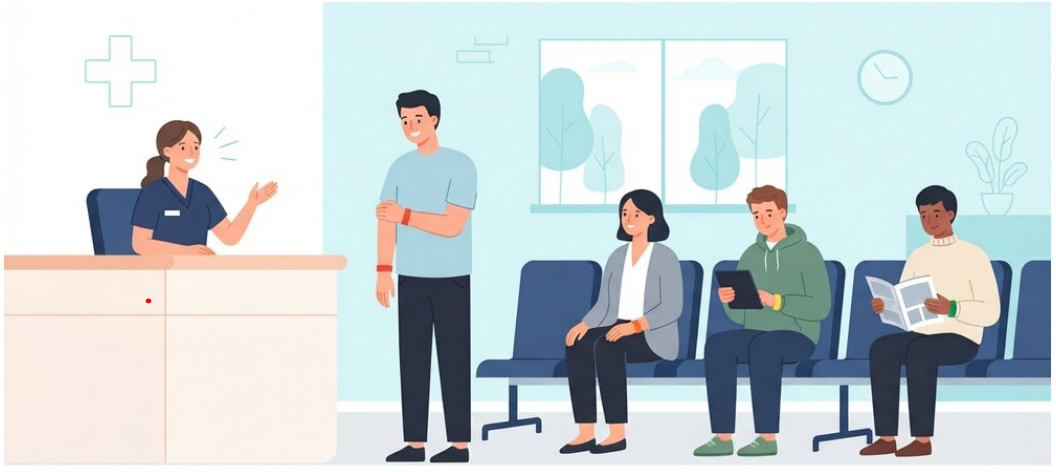
6 El contenedor `priority_queue`

7 Para el juez

8 Ejercicios

9 Cierre

La sala de urgencias



Pregunta

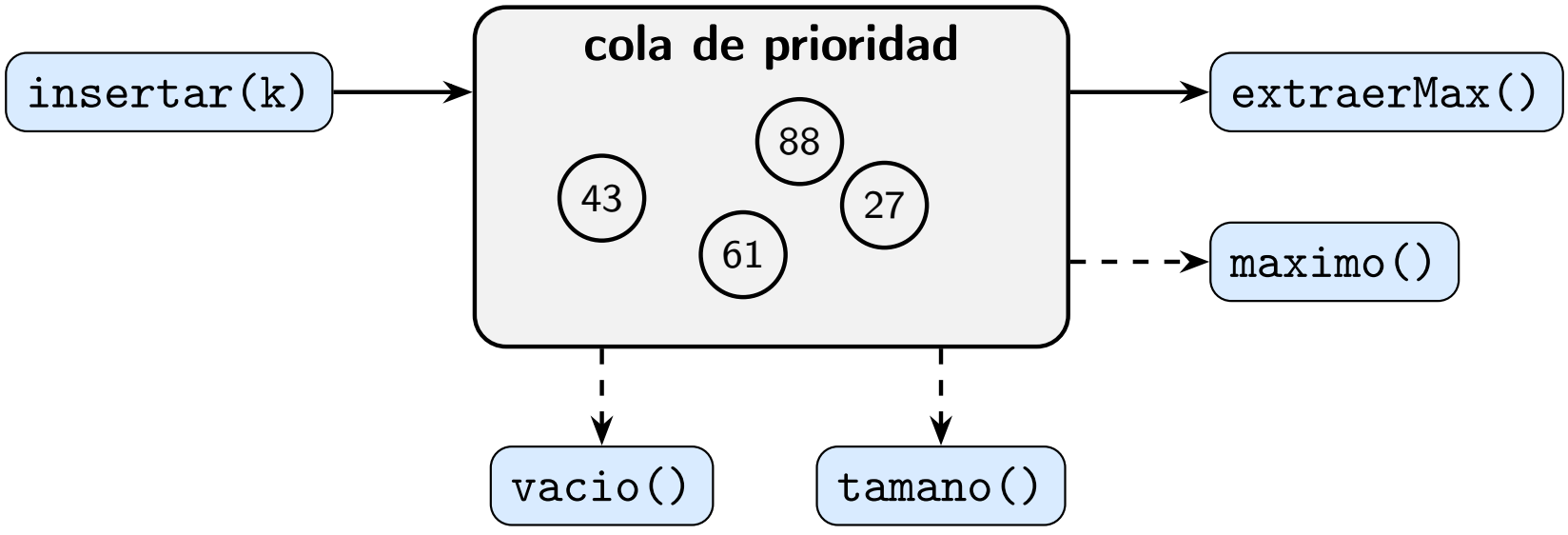
Cada paciente recibe al llegar una gravedad de 1 a 5. ¿Sirve una cola, que atiende por orden de llegada?

La sala de urgencias

Respuesta

No: un paciente grave que llega último esperaría a todos los leves. Se atiende primero al de mayor gravedad, sin importar cuándo llegó. Hace falta una cola en la que sale primero la clave de mayor prioridad: una cola de prioridad.

El contrato



Por dentro las claves no tienen un orden visible: el contrato solo promete cuál sale (CLRS, §6.5).

El contrato

Operación	Precondición	Resultado
`insertar(k)`	ninguna	`k` queda en la cola, aunque ya haya otra igual
`maximo()`	la cola no está vacía	devuelve la clave mayor, sin quitarla
`extraerMax()`	la cola no está vacía	quita la clave mayor y la devuelve
`vacio()`, `tamano()`	ninguna	si hay claves y cuántas

Una cola de prioridad de mínimos es igual con la clave menor: `minimo()` y `extraerMin()`.

El TAD en uso, con el montículo de la clase

operación	devuelve	claves en la cola
`insertar(61)`		61
`insertar(27)`		61, 27
`maximo()`	61	61, 27

`maximo()` consulta y no quita: el 61 sigue en la cola.

El TAD en uso, con el montículo de la clase

operación	devuelve	claves en la cola
`insertar(61)`		61
`insertar(27)`		61, 27
`maximo()`	61	61, 27
`insertar(88)`		88, 61, 27
`insertar(43)`		88, 61, 43, 27
`extraerMax()`	88	61, 43, 27

El 88 entró de tercero y sale primero.

El TAD en uso, con el montículo de la clase

operación	devuelve	claves en la cola
`insertar(61)`		61
`insertar(27)`		61, 27
`maximo()`	61	61, 27
`insertar(88)`		88, 61, 27
`insertar(43)`		88, 61, 43, 27
`extraerMax()`	88	61, 43, 27
`extraerMax()`	61	43, 27
`insertar(70)`		70, 43, 27
`tamano()`	3	70, 43, 27

Las claves se listan de mayor a menor para leerlas. En el montículo están en otro orden, y `uso_cola.cpp` lo reproduce con la clase `Monticulo`.

Contenido

1 Repaso: el montículo de máximos

2 Construir un montículo

3 Heapsort

4 El montículo en la STL

5 El TAD cola de prioridad

6 El contenedor `priority_queue`

7 Para el juez

8 Ejercicios

9 Cierre

priority_queue en C++

```cpp
// push, top, pop, size y empty: sale siempre el mayor.
void enteros() {
  priority_queue<int> pq;
  int llegan[5] = {64, 15, 90, 37, 52};
  int i = 0;
  while (i < 5) {
    pq.push(llegan[i]);
    i = i + 1;
  }
  printf("top() = %d, size() = %d\n", pq.top(), (int) pq.size());
  pq.pop();
  printf("tras pop(): top() = %d\n", pq.top());
  printf("salen:");
  while (!pq.empty()) {
    printf(" %d", pq.top());
    pq.pop();
  }
  printf("\n");
}
```

priority_queue en C++

Con `#include <queue>`. Guarda un `vector`: `push` es `push_back` y `push_heap`, y `pop` es `pop_heap` y `pop_back`. El programa imprime:

```
top() = 90, size() = 5
tras pop(): top() = 64
salen: 64 52 37 15
```

pop() no devuelve nada

`int x = pq.pop();` no compila: `pop` es `void`. Primero se lee con `top()` y después se saca con `pop()`. Y las dos exigen que la cola no esté vacía.

El menor arriba

El tercer parámetro de la plantilla es un `struct` comparador, como los que se usaron con `map`. Dice cuándo `a` va detrás de `b`: con `a > b`, el mayor va detrás y arriba queda el menor.

```
// a va detrás de b cuando a es mayor: arriba queda el menor.
struct MenorArriba {
  bool operator()(int a, int b) const {
    return a > b;
  }
};
```

El menor arriba

```
void minimos() {
  priority_queue<int, vector<int>, MenorArriba> pq;
  int llegan[5] = {71, 55, 83, 9, 55};
  int i = 0;
  while (i < 5) {
    pq.push(llegan[i]);
    i = i + 1;
  }
  printf("salen:");
  while (!pq.empty()) {
    printf(" %d", pq.top());
    pq.pop();
  }
  printf("\n");
}
```

9, 55, 55, 71, 83

Con 71 55 83 9 55 sale 9 55 55 71 83: un montículo de mínimos, que guarda el 55 dos veces.

Pacientes con operator<

```cpp
// Un paciente: más gravedad se atiende antes y,
// con la misma gravedad, el que llegó primero.
struct Paciente {
  int llegada;
  int gravedad;

  bool operator<(const Paciente &o) const {
    bool menos;
    if (gravedad != o.gravedad) {
      menos = gravedad < o.gravedad;
    } else {
      menos = llegada > o.llegada;
    }
    return menos;
  }
};
```

`a < b` quiere decir que `a` es menos urgente: sale después.

Pacientes con `operator<`

Llegan cinco pacientes con gravedades `3 1 4 3 4`:

```
atender: llegada 3, gravedad 4
atender: llegada 5, gravedad 4
atender: llegada 1, gravedad 3
atender: llegada 4, gravedad 3
atender: llegada 2, gravedad 1
```

Entre los de gravedad 4 sale primero el 3, que llegó antes. Sin el desempate por llegada, el orden entre iguales lo decidiría la biblioteca.

Lo que cuesta cada operación de `priority_queue`

Operación	Costo	Por qué
`push(k)`	$O(\log n)$	`push_back` y `push_heap`
`pop()`	$O(\log n)$	`pop_heap` y `pop_back`
`top()`, `size()`, `empty()`	$\Theta(1)$	`top` lee la casilla 0
`pq(v.begin(), v.end())`	$\Theta(n)$	copia el rango y llama `make_heap`
espacio	$\Theta(n)$	las claves en el `vector`, sin punteros

No tiene iteradores: para ver todas las claves hay que sacarlas.

Contenido

1 Repaso: el montículo de máximos

2 Construir un montículo

3 Heapsort

4 El montículo en la STL

5 El TAD cola de prioridad

6 El contenedor `priority_queue`

7 Para el juez

8 Ejercicios

9 Cierre

UVa 10954, *Add All*

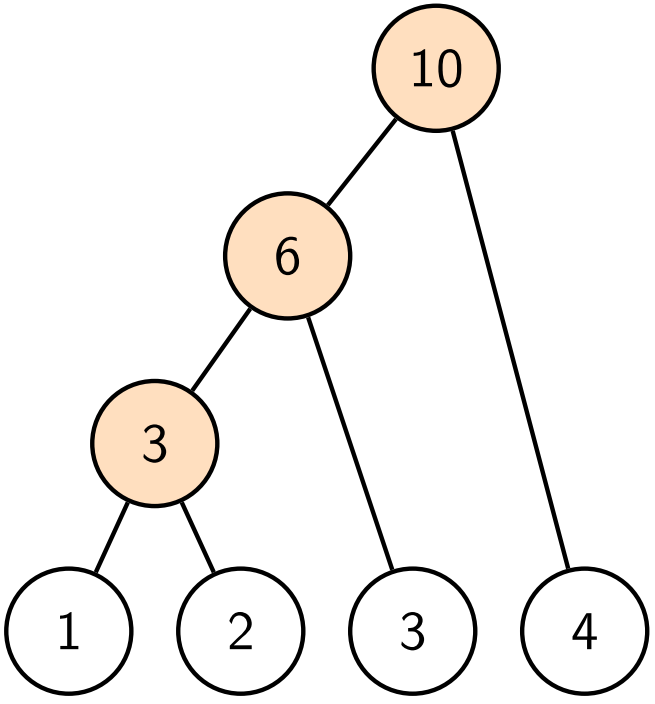
Enunciado y envío: `https://vjudge.net/problem/UVA-10954` (la página pide cuenta).

Qué pide

Sumar dos números cuesta su suma. Llegan N enteros positivos, con $2 \leq N \leq 5000$, todos menores que 100 000; hay que sumarlos todos, de a dos, con el menor costo total. Los casos terminan con $N = 0$.

Para 1 2 3: sumar $1 + 2$ cuesta 3 y luego $3 + 3$ cuesta 6, en total 9. Empezando por $2 + 3$ el total es 11.

UVa 10954: estrategia



costo 10, total 19

costo 6

costo 3

Se suman siempre los dos menores que quedan, y la suma vuelve al grupo. Una cola de prioridad de mínimos los da en $O(\log N)$: dos `extraerMin` y un `insertar` por suma, $O(N \log N)$ por caso y $\Theta(N)$ de espacio.

UVa 10954: estrategia

Pregunta

¿Basta ordenar una vez y sumar de izquierda a derecha? Pruébenlo con 5 5 5 5.

UVa 10954: estrategia

Las trampas

- Ordenar una vez no basta: con `5 5 5 5` da $10 + 15 + 20 = 45$, y la cola da $10 + 10 + 20 = 40$. La suma nueva puede ser mayor que las que quedan, y tiene que volver al grupo.
- Con 5000 números de 99 999 el total pasa de 6 000 millones, más de lo que cabe en un `int`: se acumula en `long long` y se imprime con `%lld`.

UVa 10954: la lectura

```
int n;
while (scanf("%d", &n) == 1 && n != 0) {
  int i = 0;
  while (i < n) {
    int x;
    scanf("%d", &x);
    // la clave x entra a la cola de minimos
    i = i + 1;
  }
  // sumar los dos menores mientras quede mas de uno
  // e imprimir el costo con printf("%lld\n", costo)
}
```

El ciclo de afuera para en el 0 final, que no se procesa, o si la entrada se acaba antes.

Contenido

1 Repaso: el montículo de máximos

2 Construir un montículo

3 Heapsort

4 El montículo en la STL

5 El TAD cola de prioridad

6 El contenedor `priority_queue`

7 Para el juez

8 Ejercicios

9 Cierre

Propuestos para la casa

1. **Construir a mano.** Apliquen `construir` a `21 6 39 14 52 30 47 9` y dibujen el árbol después de cada llamado a `bajar`.
2. **Heapsort a mano.** Sigan `heapsort` sobre `5 13 2 25 7 17 20 8 4`, el arreglo de los ejercicios de CLRS, §6.4.
3. **De mayor a menor.** Cambien `heapsort` para que ordene de mayor a menor. ¿Qué comparaciones cambian?

Propuestos para la casa

4. **UVa 10954, Add All.** Enunciado y envío: `https://vjudge.net/problem/UVA-10954`
5. **Un paciente empeora.** Mientras espera, la gravedad de un paciente sube. ¿Qué operación le falta al montículo para reflejarlo, y cuánto costaría si se supiera en qué casilla está?

Pistas para los propuestos

1. Empieza en $i = 3$ y queda `52 21 47 14 6 30 39 9`.
2. El montículo inicial es `25 13 20 8 7 17 2 5 4`; tras la primera ronda, `20 13 17 8 7 4 2 5 | 25`.
3. Se necesita un montículo de mínimos: cambian las dos comparaciones de `bajar`.

Pistas para los propuestos

4. Cola de mínimos con `monticulo.h` cambiado a mínimos, o `priority_queue` con el comparador de la sesión.
5. Piensen en cuál de las dos, `subir` o `bajar`, sirve cuando una clave crece (CLRS, §6.5).

Contenido

1 Repaso: el montículo de máximos

2 Construir un montículo

3 Heapsort

4 El montículo en la STL

5 El TAD cola de prioridad

6 El contenedor `priority_queue`

7 Para el juez

8 Ejercicios

9 Cierre

Resumen

- `construir` baja cada casilla con hijos, desde $\lfloor n/2 \rfloor - 1$ hasta la raíz, y cuesta $\Theta(n)$; insertar una por una cuesta $O(n \log n)$.
- Heapsort cambia el máximo con `a[fin]` y baja la nueva raíz en un montículo una casilla más corto: $O(n \log n)$ en el peor caso y $\Theta(1)$ de espacio aparte.
- La cola de prioridad promete insertar, consultar y extraer la clave mayor; con un montículo cuestan $O(\log n)$, $\Theta(1)$ y $O(\log n)$, y eso es `priority_queue`.

Lo que sigue

Estructuras de datos en Python

Las estructuras de datos que trae Python y lo que cuesta cada operación, frente a los contenedores de C++ que ya se vieron.

Referencias



T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 6, *Heapsort*: §6.1, montículos; §6.3, construir un montículo; §6.4, el algoritmo heapsort; §6.5, colas de prioridad.



UVa Online Judge. Problema 10954, *Add All*: `https://vjudge.net/problem/UVA-10954`.